

```
# Multi-Agent Individual Support Module
## Machine Learning for Groups, Teams & Social Networks
```

```
---
```

```
## Extended Multi-Agent Architecture
```

```
```python
```

```
multi_agent_support_ml.py
Extended module for multiple people, groups, teams, and social networks
Uses coupled Markov chains, game theory, and network dynamics
"""
```

```
import numpy as np
import networkx as nx
from scipy import stats, linalg, sparse
from scipy.optimize import minimize
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple, Set, Callable
from collections import defaultdict, deque
from datetime import datetime, timedelta
from enum import Enum
import json
```

```
Import previous modules
from individual_support_module import (
 SystemState, ControlInput, PhysicsMapping
)
from individual_support_ml_extended import (
 DiscreteState, MarkovChainModel, PopulationLearner,
 AdaptiveIndividualLearner, LargeNumberTheory
)
```

```
=====
MULTI-AGENT STATE SPACE
=====
```

```
@dataclass
class AgentIdentity:
 """Identity and metadata for each agent"""
 agent_id: str
 name: str
 role: Optional[str] = None
 archetype: Optional[str] = None

 # Relationships
 relationships: Dict[str, str] = field(default_factory=dict) # agent_id -> relationship_type
 group_memberships: Set[str] = field(default_factory=set)

 # Individual characteristics
 baseline_pressure: float = 5.0
 baseline_margin: float = 5.0
 baseline_temperature: float = 4.0
```

```
Learning parameters
learning_rate: float = 0.1
adaptability: float = 0.5 # How quickly they adapt to others
```

```
@dataclass
class MultiAgentState:
 """
 Joint state of multiple agents
 Captures both individual states and relational dynamics
 """

 timestamp: datetime

 # Individual states
 agent_states: Dict[str, SystemState]

 # Relational states
 connection_strengths: Dict[Tuple[str, str], float] = field(default_factory=dict)
 conflict_levels: Dict[Tuple[str, str], float] = field(default_factory=dict)
 trust_levels: Dict[Tuple[str, str], float] = field(default_factory=dict)

 # Group-level states
 group_cohesion: Dict[str, float] = field(default_factory=dict)
 group_pressure: Dict[str, float] = field(default_factory=dict)

 # Network properties
 network_density: float = 0.0
 clustering_coefficient: float = 0.0

 def __post_init__(self):
 """Compute derived network metrics"""
 n_agents = len(self.agent_states)
 if n_agents > 1:
 n_connections = len(self.connection_strengths)
 max_connections = n_agents * (n_agents - 1) / 2
 self.network_density = n_connections / max_connections if max_connections > 0 else 0.0
```

```
class RelationshipType(Enum):
 """Types of relationships between agents"""
 PARTNER = "partner"
 FAMILY = "family"
 FRIEND = "friend"
 COLLEAGUE = "colleague"
 MANAGER_REPORT = "manager_report"
 TEAMMATE = "teammate"
 MENTOR_MENTEE = "mentor_mentee"
 ADVERSARIAL = "adversarial"
```

```
=====
SOCIAL NETWORK DYNAMICS
=====
```

```
class SocialNetwork:
 """
 Models the social network structure and dynamics
```

"""

```
def __init__(self):
 self.graph = nx.Graph()
 self.agents: Dict[str, AgentIdentity] = {}

 # Network dynamics parameters
 self.influence_decay = 0.7 # How influence decays with distance
 self.contagion_rate = 0.3 # Rate of emotional contagion

def add_agent(self, agent: AgentIdentity):
 """Add agent to network"""
 self.agents[agent.agent_id] = agent
 self.graph.add_node(
 agent.agent_id,
 name=agent.name,
 role=agent.role,
 archetype=agent.archetype
)

def add_relationship(
 self,
 agent1_id: str,
 agent2_id: str,
 relationship_type: RelationshipType,
 strength: float = 1.0
):
 """Add or update relationship between agents"""
 self.graph.add_edge(
 agent1_id,
 agent2_id,
 relationship=relationship_type.value,
 strength=strength,
 trust=0.5,
 conflict=0.0
)

 # Update agent relationships
 self.agents[agent1_id].relationships[agent2_id] = relationship_type.value
 self.agents[agent2_id].relationships[agent1_id] = relationship_type.value

def add_to_group(self, agent_id: str, group_id: str):
 """Add agent to a group"""
 if agent_id in self.agents:
 self.agents[agent_id].group_memberships.add(group_id)

def get_neighbors(self, agent_id: str) -> List[str]:
 """Get direct connections of an agent"""
 return list(self.graph.neighbors(agent_id))

def get_network_distance(self, agent1_id: str, agent2_id: str) -> int:
 """Get shortest path distance between two agents"""
 try:
 return nx.shortest_path_length(self.graph, agent1_id, agent2_id)
 except nx.NetworkXNoPath:
 return float('inf')
```

```

def compute_centrality_measures(self) -> Dict[str, Dict[str, float]]:
 """Compute centrality for each agent"""
 if len(self.graph.nodes()) < 2:
 return {}

 degree_centrality = nx.degree_centrality(self.graph)
 betweenness_centrality = nx.betweenness_centrality(self.graph)

 try:
 eigenvector_centrality = nx.eigenvector_centrality(self.graph, max_iter=1000)
 except:
 eigenvector_centrality = {node: 0.0 for node in self.graph.nodes()}

 centrality = {}
 for agent_id in self.graph.nodes():
 centrality[agent_id] = {
 'degree': degree_centrality.get(agent_id, 0.0),
 'betweenness': betweenness_centrality.get(agent_id, 0.0),
 'eigenvector': eigenvector_centrality.get(agent_id, 0.0)
 }

 return centrality

def detect_communities(self) -> Dict[str, int]:
 """Detect communities using modularity optimization"""
 try:
 communities = nx.community.greedy_modularity_communities(self.graph)
 agent_to_community = {}
 for i, community in enumerate(communities):
 for agent_id in community:
 agent_to_community[agent_id] = i
 return agent_to_community
 except:
 return {agent_id: 0 for agent_id in self.graph.nodes()}

def compute_emotional_contagion(
 self,
 source_agent_id: str,
 source_temperature: float
) -> Dict[str, float]:
 """
 Compute how emotional state spreads through network
 Uses diffusion model on graph
 """
 contagion_effects = {}

 for target_agent_id in self.graph.nodes():
 if target_agent_id == source_agent_id:
 continue

 distance = self.get_network_distance(source_agent_id, target_agent_id)

 if distance == float('inf'):
 contagion_effects[target_agent_id] = 0.0
 else:
 # Contagion decays exponentially with distance
 effect = (

```

```

 self.contagion_rate *
 source_temperature *
 (self.influence_decay ** distance)
)
 contagion_effects[target_agent_id] = effect

```

```

return contagion_effects

```

```

def compute_influence_matrix(self) -> np.ndarray:
 """
 Compute influence matrix W where W_ij = influence of j on i
 """
 n_agents = len(self.graph.nodes())
 agent_ids = list(self.graph.nodes())
 W = np.zeros((n_agents, n_agents))

 for i, agent_i in enumerate(agent_ids):
 neighbors = self.get_neighbors(agent_i)

 for j, agent_j in enumerate(agent_ids):
 if agent_j in neighbors:
 # Influence based on connection strength
 edge_data = self.graph.get_edge_data(agent_i, agent_j)
 W[i, j] = edge_data.get('strength', 1.0)

 # Row-normalize (each row sums to 1)
 row_sums = W.sum(axis=1, keepdims=True)
 W = np.divide(W, row_sums, where=row_sums > 0)

 return W

```

```

=====
COUPLED MARKOV CHAIN DYNAMICS
=====

```

```

class CoupledMarkovChain:
 """
 Markov chain for multiple interacting agents
 P(s1', s2', ... | s1, s2, ..., a1, a2, ...)

 State space grows exponentially, so we use mean-field approximations
 """

 def __init__(
 self,
 n_agents: int,
 n_states_per_agent: int = 100,
 n_actions_per_agent: int = 20
):
 self.n_agents = n_agents
 self.n_states_per_agent = n_states_per_agent
 self.n_actions_per_agent = n_actions_per_agent

 # Individual models
 self.individual_models = [
 MarkovChainModel(n_states_per_agent, n_actions_per_agent)

```

```

 for _ in range(n_agents)
]

Interaction effects (pairwise)
self.interaction_effects = defaultdict(lambda: np.zeros((
 n_states_per_agent, n_states_per_agent
)))

Mean-field approximation
self.mean_field_state = np.zeros(n_agents)

def observe_joint_transition(
 self,
 agent_states: List[DiscreteState],
 agent_actions: List[int],
 next_agent_states: List[DiscreteState],
 rewards: List[float]
):
 """
 Observe a joint transition of all agents
 Updates individual models and interaction terms
 """
 # Update individual models
 for i, (state, action, next_state, reward) in enumerate(
 zip(agent_states, agent_actions, next_agent_states, rewards)
):
 self.individual_models[i].observe_transition(
 state, action, next_state, reward
)

 # Update interaction effects (pairwise)
 for i in range(self.n_agents):
 for j in range(i + 1, self.n_agents):
 # How does agent j's state affect agent i's transition?
 s_i = agent_states[i].to_index() % self.n_states_per_agent
 s_j = agent_states[j].to_index() % self.n_states_per_agent

 # Increment interaction count
 self.interaction_effects[(i, j)][s_i, s_j] += 1

def fit(self):
 """Fit all individual models"""
 for model in self.individual_models:
 model.fit()

def predict_joint_distribution(
 self,
 current_states: List[DiscreteState],
 actions: List[int]
) -> List[np.ndarray]:
 """
 Predict next state distribution for each agent
 Uses mean-field approximation: each agent sees average effect of others
 """
 next_distributions = []

 # Compute mean field

```

```

self.mean_field_state = np.array([
 s.to_index() % self.n_states_per_agent
 for s in current_states
])

for i in range(self.n_agents):
 # Base distribution from individual model
 base_dist = self.individual_models[i].predict_next_state_distribution(
 current_states[i], actions[i]
)

 # Modify based on interaction with others (mean-field)
 modified_dist = base_dist.copy()

 # Average interaction effect from all other agents
 for j in range(self.n_agents):
 if i != j and (i, j) in self.interaction_effects:
 interaction = self.interaction_effects[(i, j)]
 s_j = int(self.mean_field_state[j])

 # Add interaction effect (simplified)
 interaction_effect = interaction[:, s_j]
 interaction_effect = interaction_effect / (interaction_effect.sum() + 1e-10)
 modified_dist += 0.1 * interaction_effect # Small coupling

 # Renormalize
 modified_dist = modified_dist / modified_dist.sum()
 next_distributions.append(modified_dist)

return next_distributions

def compute_nash_equilibrium(
 self,
 current_states: List[DiscreteState]
) -> List[int]:
 """
 Compute Nash equilibrium actions for all agents
 Each agent best-responds to others' strategies

 Uses fictitious play algorithm
 """
 n_iterations = 100
 action_counts = np.ones((self.n_agents, self.n_actions_per_agent))

 for _ in range(n_iterations):
 # Each agent computes best response
 for i in range(self.n_agents):
 # Others' mixed strategies (empirical frequencies)
 other_strategies = [
 action_counts[j] / action_counts[j].sum()
 for j in range(self.n_agents) if j != i
]

 # Compute expected reward for each action
 q_values = np.zeros(self.n_actions_per_agent)
 s_i = current_states[i].to_index() % self.n_states_per_agent

```

```

for a_i in range(self.n_actions_per_agent):
 q_values[a_i] = self.individual_models[i].reward_model[s_i, a_i]

```

```

Best response
best_action = np.argmax(q_values)
action_counts[i, best_action] += 1

```

```

Return most frequent actions
equilibrium_actions = [
 np.argmax(action_counts[i])
 for i in range(self.n_agents)
]

```

```

return equilibrium_actions

```

```

=====
RELATIONSHIP DYNAMICS
=====

```

```

@dataclass

```

```

class RelationshipState:

```

```

 """State of a relationship between two agents"""

```

```

 agent1_id: str

```

```

 agent2_id: str

```

```

 relationship_type: RelationshipType

```

```

Relationship health metrics

```

```

 connection_strength: float = 1.0 # [0-1]

```

```

 trust_level: float = 0.5 # [0-1]

```

```

 conflict_level: float = 0.0 # [0-1]

```

```

Interaction patterns

```

```

 communication_frequency: float = 0.5

```

```

 positive_interactions: int = 0

```

```

 negative_interactions: int = 0

```

```

Dynamics

```

```

 trajectory: List[Tuple[datetime, float, float, float]] = field(default_factory=list)

```

```

 def update(

```

```

 self,

```

```

 delta_connection: float = 0.0,

```

```

 delta_trust: float = 0.0,

```

```

 delta_conflict: float = 0.0

```

```

):

```

```

 """Update relationship state"""

```

```

 self.connection_strength = np.clip(

```

```

 self.connection_strength + delta_connection, 0.0, 1.0

```

```

)

```

```

 self.trust_level = np.clip(

```

```

 self.trust_level + delta_trust, 0.0, 1.0

```

```

)

```

```

 self.conflict_level = np.clip(

```

```

 self.conflict_level + delta_conflict, 0.0, 1.0

```

```

)

```



```

self.trajectory.append((
 datetime.now(),
 self.connection_strength,
 self.trust_level,
 self.conflict_level
))

def get_health_score(self) -> float:
 """Overall relationship health [0-1]"""
 return (
 0.4 * self.connection_strength +
 0.4 * self.trust_level +
 0.2 * (1.0 - self.conflict_level)
)

class RelationshipDynamics:
 """
 Models dynamics of relationships between agents
 Based on Gottman's relationship research and dynamical systems theory
 """

 def __init__(self):
 self.relationships: Dict[Tuple[str, str], RelationshipState] = {}

 # Gottman ratio: 5 positive interactions to 1 negative for healthy relationship
 self.gottman_ratio_healthy = 5.0

 # Relationship differential equations parameters
 self.influence_params = {
 RelationshipType.PARTNER: {'mutual': 0.8, 'repair': 0.6},
 RelationshipType.FAMILY: {'mutual': 0.7, 'repair': 0.5},
 RelationshipType.FRIEND: {'mutual': 0.6, 'repair': 0.4},
 RelationshipType.COLLEAGUE: {'mutual': 0.4, 'repair': 0.3},
 RelationshipType.TEAMMATE: {'mutual': 0.5, 'repair': 0.4}
 }

 def add_relationship(self, relationship: RelationshipState):
 """Add or update relationship"""
 key = tuple(sorted([relationship.agent1_id, relationship.agent2_id]))
 self.relationships[key] = relationship

 def get_relationship(self, agent1_id: str, agent2_id: str) -> Optional[RelationshipState]:
 """Get relationship between two agents"""
 key = tuple(sorted([agent1_id, agent2_id]))
 return self.relationships.get(key)

 def evolve_relationship(
 self,
 relationship: RelationshipState,
 agent1_state: SystemState,
 agent2_state: SystemState,
 interaction_quality: float, # [-1, 1]
 dt: float = 1.0
) -> RelationshipState:
 """
 Evolve relationship state based on agent states and interaction

```

Based on Gottman-Murray model:

$$dC_1/dt = r_1 + \alpha_1 C_2 - \beta_1 C_1$$

$$dC_2/dt = r_2 + \alpha_2 C_1 - \beta_2 C_2$$

Where  $C_i$  is connection level,  $r_i$  is uninfluenced state,  
 $\alpha_i$  is influence from other,  $\beta_i$  is return to baseline

```
params = self.influence_params.get(
 relationship.relationship_type,
 {'mutual': 0.5, 'repair': 0.3}
)

Uninfluenced state (how each person feels independently)
r1 = -0.1 * agent1_state.strain / 10.0 # Strain reduces connection
r2 = -0.1 * agent2_state.strain / 10.0

Mutual influence
alpha = params['mutual']

Return to baseline
beta = 0.2

Interaction effect
if interaction_quality > 0:
 relationship.positive_interactions += 1
else:
 relationship.negative_interactions += 1

Compute Gottman ratio
total_interactions = relationship.positive_interactions + relationship.negative_interactions
if total_interactions > 0:
 gottman_ratio = relationship.positive_interactions / max(relationship.negative_interactions, 1)
 relationship_health_trend = (gottman_ratio - self.gottman_ratio_healthy) /
self.gottman_ratio_healthy
else:
 relationship_health_trend = 0.0

Differential equations (simplified)
dC1 = (r1 + alpha * relationship.connection_strength - beta * relationship.connection_strength) * dt
dC2 = (r2 + alpha * relationship.connection_strength - beta * relationship.connection_strength) * dt

Update connection
delta_connection = 0.5 * (dC1 + dC2) + 0.3 * interaction_quality

Trust dynamics (builds slowly, erodes quickly)
if interaction_quality > 0:
 delta_trust = 0.05 * interaction_quality # Builds slowly
else:
 delta_trust = 0.2 * interaction_quality # Erodes quickly

Conflict dynamics
conflict_generation = 0.3 * (
 agent1_state.temperature / 10.0 +
 agent2_state.temperature / 10.0
)
conflict_resolution = -0.1 * relationship.trust_level
```

```

delta_conflict = (conflict_generation + conflict_resolution) * dt

Update
relationship.update(
 delta_connection=delta_connection,
 delta_trust=delta_trust,
 delta_conflict=delta_conflict
)

return relationship

def predict_relationship_trajectory(
 self,
 relationship: RelationshipState,
 agent1_trajectory: List[SystemState],
 agent2_trajectory: List[SystemState],
 interaction_pattern: List[float], # Sequence of interaction qualities
 horizon: int = 24
) -> List[RelationshipState]:
 """Predict how relationship will evolve"""
 trajectory = [relationship]
 current_rel = relationship

 for t in range(horizon):
 a1_state = agent1_trajectory[min(t, len(agent1_trajectory) - 1)]
 a2_state = agent2_trajectory[min(t, len(agent2_trajectory) - 1)]
 interaction = interaction_pattern[min(t, len(interaction_pattern) - 1)]

 # Create copy to avoid modifying original
 next_rel = RelationshipState(
 agent1_id=current_rel.agent1_id,
 agent2_id=current_rel.agent2_id,
 relationship_type=current_rel.relationship_type,
 connection_strength=current_rel.connection_strength,
 trust_level=current_rel.trust_level,
 conflict_level=current_rel.conflict_level,
 positive_interactions=current_rel.positive_interactions,
 negative_interactions=current_rel.negative_interactions
)

 next_rel = self.evolve_relationship(
 next_rel, a1_state, a2_state, interaction, dt=1.0
)

 trajectory.append(next_rel)
 current_rel = next_rel

 return trajectory

def recommend_relationship_intervention(
 self,
 relationship: RelationshipState
) -> List[Dict]:
 """Recommend interventions to improve relationship health"""
 interventions = []

 health_score = relationship.get_health_score()

```

```

Low connection
if relationship.connection_strength < 0.4:
 interventions.append({
 'priority': 'HIGH',
 'type': 'connection_building',
 'name': 'Increase Quality Time',
 'description': 'Schedule dedicated, distraction-free time together',
 'expected_effect': 0.3,
 'time_frame': '1-2 weeks'
 })

Low trust
if relationship.trust_level < 0.4:
 interventions.append({
 'priority': 'HIGH',
 'type': 'trust_building',
 'name': 'Consistency & Reliability',
 'description': 'Follow through on commitments, be predictable',
 'expected_effect': 0.25,
 'time_frame': '2-4 weeks'
 })

High conflict
if relationship.conflict_level > 0.6:
 interventions.append({
 'priority': 'CRITICAL',
 'type': 'conflict_resolution',
 'name': 'Structured Dialogue',
 'description': 'Use active listening and "I" statements',
 'expected_effect': -0.4, # Reduces conflict
 'time_frame': 'immediate'
 })

Poor Gottman ratio
total = relationship.positive_interactions + relationship.negative_interactions
if total > 10:
 ratio = relationship.positive_interactions / max(relationship.negative_interactions, 1)
 if ratio < self.gottman_ratio_healthy:
 interventions.append({
 'priority': 'MEDIUM',
 'type': 'positive_interaction',
 'name': 'Increase Positive Interactions',
 'description': f'Current ratio {ratio:.1f}:1, target {self.gottman_ratio_healthy}:1',
 'expected_effect': 0.2,
 'time_frame': '1-2 weeks'
 })

return interventions

```

```

=====
GROUP DYNAMICS
=====

```

```

@dataclass
class GroupState:

```

```
"""State of a group of agents"""
```

```
group_id: str
```

```
name: str
```

```
member_ids: Set[str]
```

```
Group-level metrics
```

```
cohesion: float = 0.5 # [0-1]
```

```
psychological_safety: float = 0.5 # [0-1]
```

```
collective_efficacy: float = 0.5 # [0-1]
```

```
Group pressure and margin
```

```
aggregate_pressure: float = 5.0
```

```
aggregate_margin: float = 5.0
```

```
Performance
```

```
productivity: float = 0.5
```

```
innovation_rate: float = 0.3
```

```
Dynamics
```

```
formation_date: datetime = field(default_factory=datetime.now)
```

```
lifecycle_stage: str = "forming" # forming, storming, norming, performing, adjourning
```

```
class GroupDynamics:
```

```
"""
```

```
Models dynamics of groups and teams
```

```
Based on Tuckman's stages and team science
```

```
"""
```

```
def __init__(self, network: SocialNetwork):
```

```
 self.network = network
```

```
 self.groups: Dict[str, GroupState] = {}
```

```
 # Tuckman's stages parameters
```

```
 self.stage_transitions = {
```

```
 'forming': {'cohesion': 0.3, 'conflict': 0.2, 'next': 'storming'},
```

```
 'storming': {'cohesion': 0.4, 'conflict': 0.7, 'next': 'norming'},
```

```
 'norming': {'cohesion': 0.7, 'conflict': 0.3, 'next': 'performing'},
```

```
 'performing': {'cohesion': 0.9, 'conflict': 0.1, 'next': 'performing'},
```

```
 'adjourning': {'cohesion': 0.6, 'conflict': 0.2, 'next': 'adjourning'}
```

```
 }
```

```
def create_group(self, group_state: GroupState):
```

```
 """Create a new group"""
```

```
 self.groups[group_state.group_id] = group_state
```

```
 # Add group membership to all members
```

```
 for member_id in group_state.member_ids:
```

```
 if member_id in self.network.agents:
```

```
 self.network.add_to_group(member_id, group_state.group_id)
```

```
def compute_group_state(
```

```
 self,
```

```
 group_id: str,
```

```
 member_states: Dict[str, SystemState],
```

```
 relationship_states: Dict[Tuple[str, str], RelationshipState]
```

```
) -> GroupState:
```

```
"""
```

```
Compute current group state from member states
```

```
"""
```

```
group = self.groups.get(group_id)
```

```
if not group:
```

```
 return None
```

```
members = list(group.member_ids)
```

```
n_members = len(members)
```

```
if n_members == 0:
```

```
 return group
```

```
Aggregate individual metrics
```

```
pressures = [member_states[m].pressure for m in members if m in member_states]
```

```
margins = [member_states[m].margin for m in members if m in member_states]
```

```
temperatures = [member_states[m].temperature for m in members if m in member_states]
```

```
group.aggregate_pressure = np.mean(pressures) if pressures else 5.0
```

```
group.aggregate_margin = np.mean(margins) if margins else 5.0
```

```
avg_temperature = np.mean(temperatures) if temperatures else 5.0
```

```
Cohesion from relationship strengths
```

```
relationship_strengths = []
```

```
for i, m1 in enumerate(members):
```

```
 for m2 in members[i+1:]:
```

```
 key = tuple(sorted([m1, m2]))
```

```
 if key in relationship_states:
```

```
 relationship_strengths.append(
 relationship_states[key].connection_strength
```

```
)
```

```
group.cohesion = np.mean(relationship_strengths) if relationship_strengths else 0.5
```

```
Psychological safety (inverse of conflict)
```

```
conflict_levels = []
```

```
for i, m1 in enumerate(members):
```

```
 for m2 in members[i+1:]:
```

```
 key = tuple(sorted([m1, m2]))
```

```
 if key in relationship_states:
```

```
 conflict_levels.append(
 relationship_states[key].conflict_level
```

```
)
```

```
avg_conflict = np.mean(conflict_levels) if conflict_levels else 0.0
```

```
group.psychological_safety = 1.0 - avg_conflict
```

```
Collective efficacy (function of cohesion, margin, and low pressure)
```

```
group.collective_efficacy = (
```

```
 0.4 * group.cohesion +
```

```
 0.3 * (group.aggregate_margin / 10.0) +
```

```
 0.3 * (1.0 - group.aggregate_pressure / 10.0)
```

```
)
```

```
Productivity (Yerkes-Dodson: inverted U with pressure)
```

```
optimal_pressure = 5.0
```

```
pressure_factor = 1.0 - abs(group.aggregate_pressure - optimal_pressure) / 10.0
```

```

group.productivity = (
 0.5 * group.collective_efficacy +
 0.3 * pressure_factor +
 0.2 * group.psychological_safety
)

Innovation (requires safety and moderate pressure)
group.innovation_rate = (
 0.5 * group.psychological_safety +
 0.3 * min(group.aggregate_pressure / 10.0, 0.7) +
 0.2 * group.cohesion
)

Update lifecycle stage
self._update_lifecycle_stage(group)

return group

def _update_lifecycle_stage(self, group: GroupState):
 """Update group lifecycle stage (Tuckman's model)"""
 current_stage = group.lifecycle_stage
 stage_params = self.stage_transitions.get(current_stage, {})

 target_cohesion = stage_params.get('cohesion', 0.5)

 # Check if ready to advance

```